

Open RAN em Kubernetes

Arquitetura implantada, Nephio, xApps KPM e laboratório multi-UE

Estado validado no NMI e no CIC

POC acadêmica em dois clusters: cinco nós amd64 no NMI e dois nós ARM64 no CIC. Open5GS, RAN, Near-RT RIC, Nephio/Porch, Flux, três xApps KPM e três perfis de UE foram implantados e validados pela VPN institucional do NMI.

Documento para apresentação técnica e planejamento de evolução

Versão 2.0 • 31 de agosto de 2026

1. Resumo executivo

A pilha demonstra uma POC Open RAN virtualizada, multi-host, multi-site e multiarch; o próximo passo é transformar esse ambiente em laboratório guiado para alunos.

Mensagem central

Kubernetes executa e conecta workloads. O Near-RT RIC hospeda xApps de observação/controle da RAN. Nephio/Porch especializa pacotes por site e arquitetura; Flux os reconcilia nos clusters NMI e CIC. São camadas complementares.

O que existe hoje

- Cluster NMI com cinco nós amd64: VM 105 como control plane; VM 111, VM 112, VM 113 e nmi-srv03 como workers.
- Cluster CIC com dois nós ARM64 Ampere e r4-simple-mon implantada por Flux a partir da mesma imagem multiarch usada no NMI.
- Open5GS como 5G Core, OCUDU como CU/DU, srsUE como UE virtual e O-RAN SC M-release como Near-RT RIC.
- Flannel + Multus/OVS para rede; Prometheus, Grafana, Loki, Alertmanager e Alloy para métricas, logs e alertas.
- Nephio/Porch e Web UI executam no próprio cluster NMI, em namespaces separados e com afinidade para a VM 113.
- Acesso institucional pela VPN do NMI; Tailnet é apenas canal auxiliar de manutenção e não integra a arquitetura apresentada.

Próximo experimento recomendado

- Um aluno cria uma nova xApp a partir do scaffold didático e implementa sua lógica de análise.
- A imagem é publicada em amd64/arm64, empacotada no Porch e entregue por Flux no NMI; a variante ARM é testada no CIC.
- UE2 ou UE3 gera tráfego real pelo UPF, a xApp muda de estado e a evidência aparece no Prometheus/Grafana.

2. Topologia implantada e laboratórios

O NMI combina dois Proxmox Intel e um Dell físico; o CIC já executa o workload ARM64 em duas Ampere.

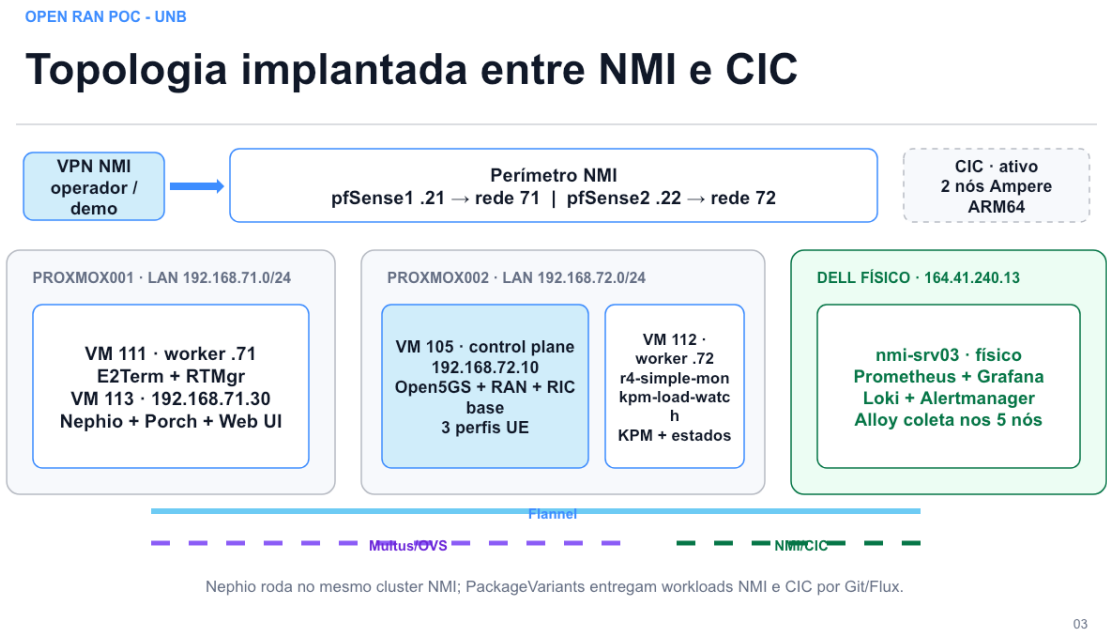


Figura 1 — Topologia física e lógica implantada entre NMI e CIC.

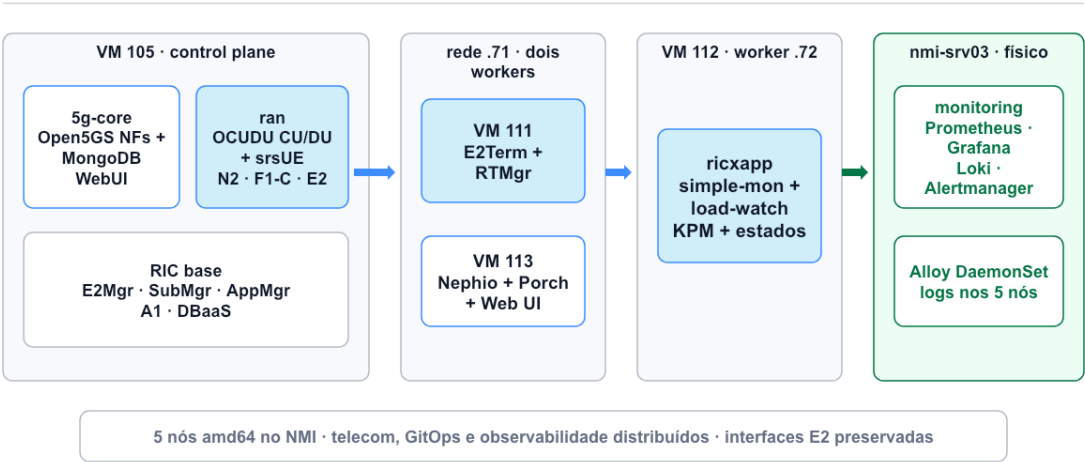
Local	Infraestrutura	Redes/uso	Situação
Proxmox001	VM 111 + VM 113	192.168.71.20/.30	E2Term/RTMgr + Nephio/Porch
Proxmox002	VM 105 + VM 112	192.168.72.10/.20; pod CIDRs 10.244.0.0/24 e 10.244.3.0/24	Control plane/Core/RAN + xApp
NMI físico	nmi-srv03 / Ubuntu 24.04	164.41.240.13; pod CIDR 10.244.1.0/24	Observabilidade persistente
CIC	2 × nós Kubernetes Ampere	Cluster ARM64 acessível pela rede do laboratório	r4-simple-mon + Flux Ready
Gestão	VPN do NMI + pfSense	rede 71 via .21; rede 72 via .22	Acesso institucional e regras restritas

3. Arquitetura de software atual

A POC reúne 5G Core, RAN virtual, Near-RT RIC, Nephio/GitOps e observabilidade.

OPEN RAN POC - UNB

NMI: telecom, Nephio e observabilidade



04

Figura 2 — Componentes implantados e interfaces principais da pilha.

Domínio	Principais componentes	Responsabilidade
5G Core	Open5GS + MongoDB + WebUI	Registro, autenticação, sessões e plano de usuário
RAN	OCUDU CU/DU + srsUE + ZMQ	Emulação da gNB e do terminal sem rádio físico
Near-RT RIC	E2Term, E2Mgr, SubMgr, RTMgr, AppMgr, A1, RMR	Conectar nós E2 e hospedar xApps
Automação	Nephio Web UI + Porch + Gitea + Flux	Pacotes, variantes por site e reconciliação GitOps
Rede K8s	Flannel + Multus + OVS-CNI	Rede de pods e interfaces Open RAN dedicadas

4. Como a pilha funciona de ponta a ponta

Há dois caminhos que precisam ser entendidos separadamente: tráfego do usuário e telemetria/controle da RAN.

Plano de usuário do UE

1. srsUE conecta-se ao DU por RF virtual ZMQ e obtém endereço na rede 10.45.0.0/24 em tun_srsue.
2. DU/CU processam a pilha NR; o CU-CP conversa com o AMF pela interface N2.
3. O CU-UP encaminha o plano de usuário até o UPF; o UPF aplica roteamento/NAT quando o destino é externo.

Telemetria E2/KPM

1. OCUDU estabelece SCTP/E2AP com o E2Term através da rede e2 (10.200.3.0/24).
2. E2Mgr mantém o inventário dos nós e funções RAN anunciadas.
3. Uma xApp solicita ao SubMgr uma assinatura E2SM-KPM.
4. RTMgr/RMR direcionam a indicação recebida pelo E2Term até a xApp inscrita.
5. A xApp decodifica o ASN.1 e registra a métrica KPM selecionada.

Estado do egress

Validado em 31/08/2026: UE2 e UE3 registraram sequencialmente no Open5GS, receberam endereços 10.45.0.x, transferiram 50 MB com HTTP 200 e produziram KPM real. NAT, encaminhamento e retorno do UPF estão funcionais.

5. Near-RT RIC: quem faz o quê

O RIC não substitui o cluster; ele é uma plataforma de controle quase em tempo real para a RAN.

Componente	Função prática	Interface/fluxo
E2Term	Termina SCTP/E2AP e traduz mensagens para o barramento interno	E2 ↔ RMR
E2Mgr	Mantém inventário e estado de nós E2	REST + banco
SubMgr	Cria e acompanha subscriptions E2	REST + RMR
RTMgr	Gera/atualiza rotas do barramento	Tabelas RMR
RMR	Transporta mensagens entre componentes e xApps	mtypes + rotas
AppMgr	Registra e gerencia xApps	REST
A1 Mediator	Recebe políticas do plano não-RT	A1

Estado observado

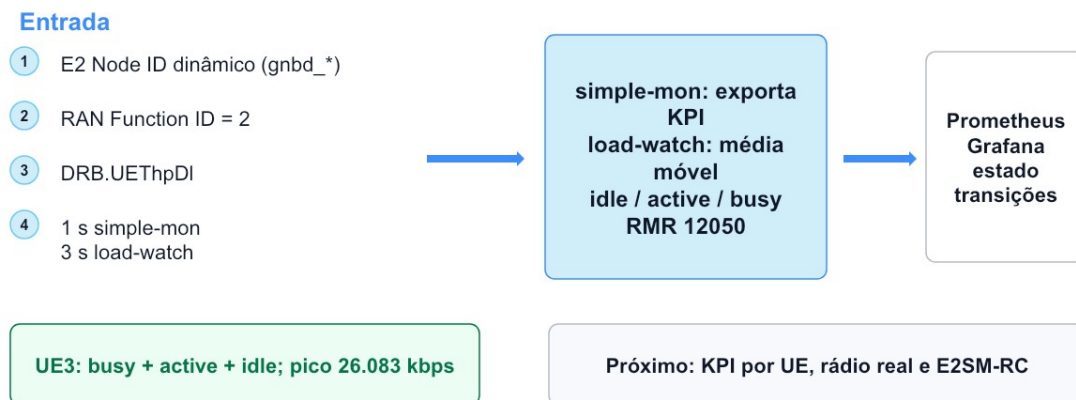
- O E2 Node ID é dinâmico (gnbd_*) e o nó ativo permanece CONNECTED no inventário.
- As xApps r4-simple-mon e kpm-load-watch estão registradas no NMI, mantêm assinaturas independentes e recebem indicações KPM via RMR mtype 12050.
- E2Term e RTMgr executam na VM 111, enquanto a xApp executa na VM 112; o E2Term permaneceu estável durante os rollouts.

6. As xApps KPM do projeto

simple-mon exporta o KPI recebido; load-watch interpreta uma média móvel e publica estados idle, active e busy.

OPEN RAN POC - UNB

Duas xApps monitoram KPM; não controlam a RAN



07

Figura 3 — Ciclo de vida da xApp, do registro até a indicação KPM.

Implementação utilizada

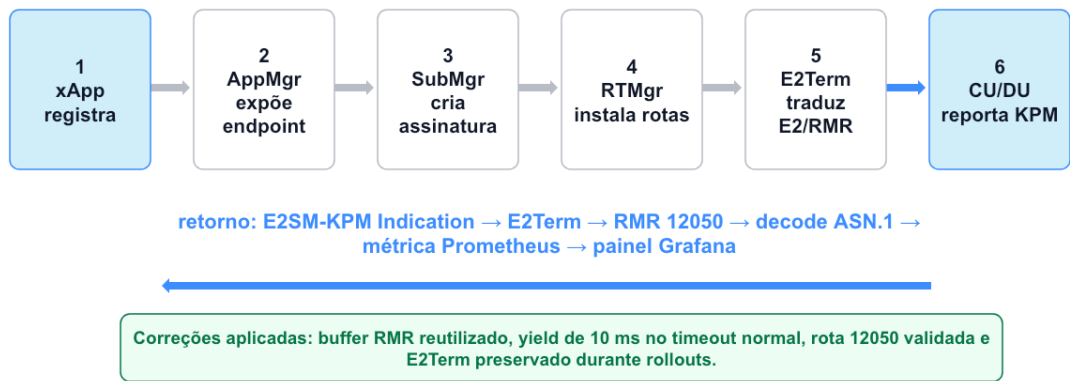
- Entrada efetiva do chart: xapps/python/kpm_mon_xapp.py.
- Estilo de relatório: 1; função RAN: 2; métrica: DRB.UETpDI.
- Mensagem RMR esperada para indicação: mtype 12050.
- Decodificação E2SM-KPM/ASN.1, publicação da série em /metrics e scrape pelo Prometheus.
- kpm-load-watch reutiliza a mesma assinatura KPM e adiciona estado, transições e média móvel para uma demonstração pedagógica.

7. Limites atuais das xApps

A distinção entre observação e controle evita prometer um closed loop que o código ainda não implementa.

OPEN RAN POC - UNB

Fluxo KPM pela interface E2



08

Figura 4 — Fluxo da indicação E2 até a métrica interpretada pela xApp.

Capacidade	Hoje	Evolução
Registro no RIC	Validado no AppMgr	Empacotar declarativamente
Subscription KPM	Ativa e validada pelo Ansible	Gerenciar por GitOps
Receber/decodificar KPM	NMI e CIC validados; 1.000+ indicações no ARM	Ampliar catálogo de KPIs
Decisão de controle	Não implementada	Definir política e limiares
Ação sobre RAN	Não implementada	E2 Control/A1 após segurança e testes
Detalhe operacional A correção reutiliza o buffer de recepção RMR, adiciona yield de 10 ms no timeout normal, valida a rota 12050 e evita reiniciar o E2Term durante rollouts, preservando o SCTP com o DU.		

8. Estado validado nos clusters NMI e CIC

A fotografia abaixo foi obtida por validação funcional em 31 de agosto de 2026.

Item	Estado	Evidência/observação
Kubernetes	OK	NMI 5/5 amd64 Ready; CIC 2/2 ARM64 Ready
Placement	OK	Telecom/Nephio/observabilidade no NMI; xApp ARM64 no CIC
Rede inter-host	OK	Flannel + OVS VXLAN full mesh; 0% de perda
Open5GS WebUI	OK	HTTP 200 pela VPN em http://192.168.72.10:30454
E2 + xApp	OK	2 xApps no NMI + 1 no CIC; assinaturas independentes
UE	OK	três perfis; UE2/UE3 testados em sequência
Internet via UE	OK	50 MB pela tun_srsue; HTTP 200 para UE2 e UE3
KPM	OK	load-watch transitou idle/active/busy; pico médio 26.083 kbps
Nephio/GitOps	OK	3 PackageVariants; Flux NMI/CIC Ready
Observabilidade	OK	Prometheus/Grafana/Loki/Alertmanager persistentes
Grafana	OK	VPN: http://192.168.72.10:30300

Roteiro de demonstração recomendado

1. Mostrar 5 nós Ready no NMI e 2 nós ARM64 Ready no CIC.
2. Abrir a WebUI pela VPN do NMI e exibir os três assinantes.
3. Abrir Nephio/Porch e mostrar as variantes NMI, CIC e load-watch.
4. Executar scripts/demo-ue-lab.sh ue2 ou ue3 e acompanhar os 50 MB via tun_srsue.
5. Correlacionar idle → active/busy → idle no Grafana e mostrar Flux Ready nos dois sites.

9. Redes e versões

A repetibilidade depende de manter interfaces, versões e exceções de arquitetura documentadas.

Rede/interface	Faixa/endpoint	Uso
Rede primária K8s	Flannel / pod CIDRs por nó	Comunicação padrão entre pods via VXLAN UDP/4789
n2	10.200.1.0/24	CU-CP ↔ AMF
f1c	10.200.2.0/24	CU-CP ↔ DU
e2	10.200.3.0/24	OCUDU ↔ E2Term
UE tunnel	10.45.0.0/24	Plano de usuário do srsUE
NMI rede 71	192.168.71.0/24	VM 111 em .20 e VM 113 em .30; via pfSense1 .21
NMI rede 72	192.168.72.0/24	VM 105 em .10 e VM 112 em .20; via pfSense2 .22
VPN NMI	acesso às redes 71/72	Gestão institucional, WebUI, Grafana e SSH

Baseline de software

Camada	Versão
Open5GS / MongoDB	2.7.7 / 8.0.29
OCUDU / srsUE	release_26_04 / release_25_10
O-RAN SC	M-release; e2/e2mgr 6.0.8, appmgr 0.5.10, submgr 0.10.4, rtmgr 0.9.7
Kubernetes atual	1.30.14

10. Limites assumidos e agenda de pesquisa

O ambiente é laboratório acadêmico, não produção: simplicidade, repetibilidade e liberdade experimental têm prioridade sobre alta disponibilidade.

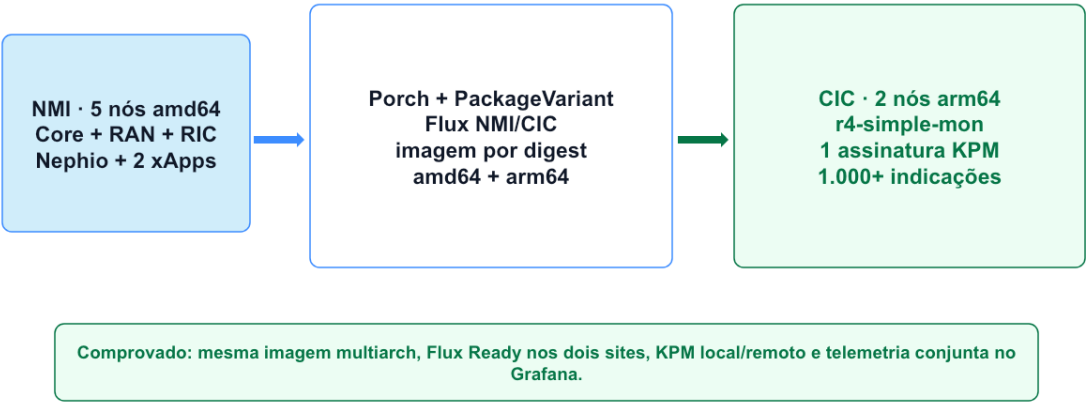
Risco/lacuna	Impacto	Ação proposta
Control plane único	Falha interrompe a demonstração	Aceitar no laboratório; documentar recuperação
Um peer ZMQ	UEs simulados entram em sequência	Adicionar peers/rádio conforme o experimento
KPM agregado	xApp vê carga do nó, não o IMSI	Explorar Report Styles por UE
Rota RMR sensível	Rollout incorreto pode perder KPM	Manter gates automatizados e não reiniciar E2Term
SIM físico ausente	Teste ainda não usa aparelho real	Usar K/OPc exclusivos com rádio e SIM programável
Sem ação de controle	xApps apenas observam	Experimentar E2SM-RC com guardrails
Logs curtos	Histórico não é permanente	Adequado à POC; exportar só evidência de artigos
Princípio do laboratório Cada experimento deve ser reproduzível, observável e reversível. Não é necessário simular requisitos de produção que não contribuem para ensino ou pesquisa.		

11. Arquitetura de infraestrutura implantada

A distribuição usa as máquinas dos dois laboratórios sem confundir demonstração multi-host com alta disponibilidade completa.

OPEN RAN POC - UNB

NMI e CIC já executam o mesmo pacote multiarch



13

Figura 5 — O mesmo pacote multiarch reconciliado no NMI e no CIC.

Nó	Hospedagem	Papel atual	Workloads principais
oran-k8s-01 / VM 105	proxmox002	Control plane	Core, RAN e RIC base
oran-k8s-w01 / VM 111	proxmox001	Worker	E2Term e RTMgr
oran-k8s-w02 / VM 112	proxmox002	Worker	r4-simple-mon + kpm-load-watch
nephio-k8s-w01 / VM 113	proxmox001	Worker	Porch, Web UI, Gitea e controllers
nmi-srv03	Dell físico	Worker de observabilidade	Prometheus, Grafana, Loki e Alertmanager
cic-k8s-cp01 + cic-k8s-w01	Ampere CIC	Cluster ARM64	Flux + r4-simple-mon
Gate de rede concluído O nmi-srv03 mantém rotas para 192.168.71.0/24 via 164.41.240.21 e para 192.168.72.0/24 via 164.41.240.22. Flannel e OVS formam malha entre os hosts com regras WAN restritas.			

11.1 Observabilidade implantada

O servidor físico concentra armazenamento e visualização, enquanto o Alloy coleta somente os namespaces relevantes nos cinco nós NMI e o Grafana agrega a telemetria CIC.

Componente	Persistência/retenção	Função e acesso
Prometheus	20 GiB; 7 dias; limite 16 GB	Métricas da pilha, xApp e Kubernetes
Grafana	5 GiB	http://192.168.72.10:30300 pela VPN NMI
Alertmanager	2 GiB; 5 dias	http://192.168.72.10:30301 pela VPN NMI
Loki	20 GiB; retenção 24 h	Logs selecionados da pilha
Alloy	DaemonSet nos 5 nós NMI	5g-core, near-rt-ric, ricxapp, ran e monitoring

Dashboards e alertas

- O-RAN Stack Overview: saúde do cluster, E2, xApp, KPM e UE.
- O-RAN Kubernetes Logs: investigação por namespace, pod e nível.
- Nove regras customizadas cobrem nó, crash loop, E2Term, xApp, UE, KPM stale, RMR e storage.
- Logs de scrapes/KPM/RLC bem-sucedidos são descartados antes do Loki; ociosidade do UE não gera volume artificial.

Operação de laboratório

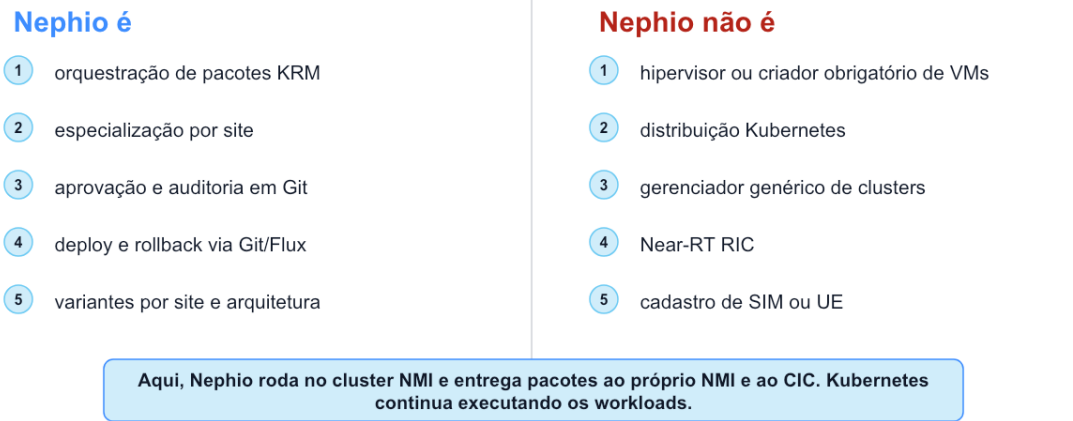
A retenção é deliberadamente curta: o objetivo é demonstrar e diagnosticar a POC, não acumular logs de teste.

12. Onde o Nephio entra

Nephio é uma camada de automação GitOps para o ciclo de vida de funções de rede em clusters Kubernetes.

OPEN RAN POC - UNB

Nephio complementa Kubernetes



14

Figura 6 — Responsabilidades: infraestrutura, Kubernetes, RIC e Nephio não são a mesma coisa.

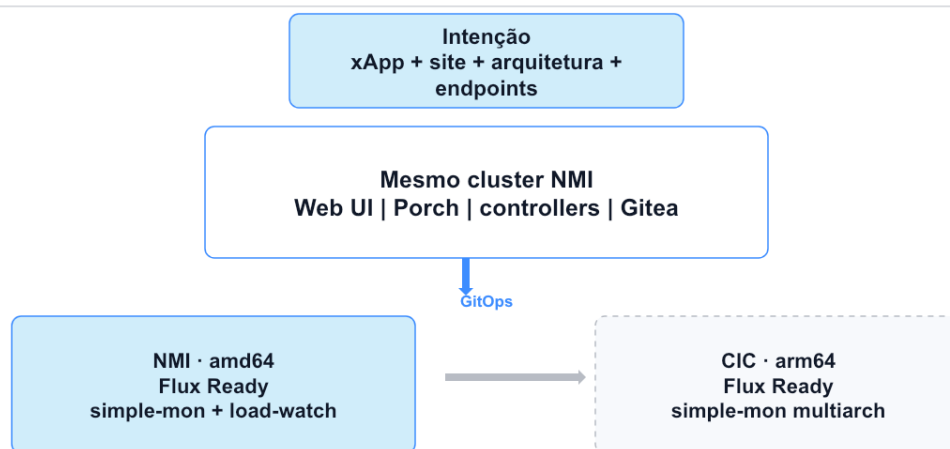
Ferramenta/camada	Papel	Não é
Proxmox	VMs, storage e bridges	Orquestrador de NFs
Kubernetes	Pods, serviços, volumes e scheduling	Gerência semântica da rede 5G
Rancher/OpenShift	Gestão/plataforma de clusters	Automação telco específica do Nephio
Near-RT RIC	xApps e controle/observação da RAN	Gerenciador de clusters
Nephio	Pacotes, especialização e implantação declarativa de NFs	Substituto das camadas anteriores

13. Nephio implantado e fluxo GitOps

Na POC, os componentes Nephio executam no próprio cluster NMI, isolados por namespaces e afinidade; NMI e CIC são alvos de pacotes.

OPEN RAN POC - UNB

Nephio já entrega variantes NMI e CIC via GitOps



15

Figura 7 — Porch especializa variantes e Flux reconcilia NMI amd64 e CIC arm64.

Camada de gestão no NMI

- Hospeda Porch, controladores, Web UI e Gitea em namespaces separados, preferencialmente na VM 113.
- Mantém catálogos e PackageVariants; para esta POC não há benefício em operar um segundo cluster Kubernetes apenas para gestão.

Workload clusters

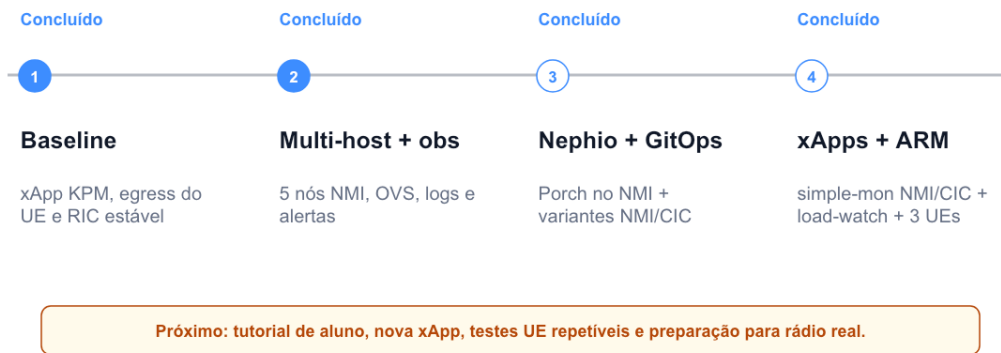
- NMI recebe simple-mon e load-watch por Flux, junto da pilha telecom já validada.
- CIC recebe a variante ARM64 do simple-mon a partir da mesma imagem multiarch fixada por digest.
- PackageVariants kpm-load-watch-nmi, simple-mon-nmi-v5 e simple-mon-cic-arm64 permanecem publicados.

14. Marcos concluídos e próximo ciclo

A infraestrutura, o GitOps e a variante ARM já foram comprovados; o próximo ciclo é didático e orientado a experimentos.

OPEN RAN POC - UNB

Seis marcos concluídos; o próximo é didático



16

Figura 8 — Seis marcos concluídos; o próximo é o laboratório guiado.

Fase	Entregável	Critério de aceite
0 — Baseline	xApp, egress e RIC estável	Concluído
1 — Workers	Uma VM em cada Proxmox	Concluído: scheduling e rede inter-host
2 — Observabilidade	Persistência, logs e alertas	Concluído: PVCs, Loki, Alloy e Alertmanager
3 — Nephio	Porch + Web UI + GitOps no NMI	Concluído: variantes publicadas
4 — Pacotes xApp	simple-mon + load-watch	Concluído: deploy e rollback via Git/Flux
5 — CIC ARM	Cluster Ampere	Concluído: imagem multiarch + KPM
6 — Multi-UE	Perfis UE1/UE2/UE3	Concluído: attach, HTTP 200 e KPM
7 — Alunos	Nova xApp + tutorial	Aluno publica, implanta e explica a evidência

15. Decisões técnicas consolidadas

A POC adotou um único caminho GitOps e evitou criar infraestrutura sem benefício didático.

Compatibilidade crítica

Nephio R6 declara suporte a Kubernetes 1.26–1.32. O NMI em 1.30 está dentro da faixa. Uma baseline futura em 1.36 não deve hospedar R6 sem validação ou atualização do Nephio.

Escolhas recomendadas

- Nephio/Porch no mesmo cluster NMI, isolado por namespaces e afinidade na VM 113.
- Flux como único reconciliador GitOps dos workloads NMI e CIC.
- r4-simple-mon como pacote-base; kpm-load-watch como evolução didática com estado observável.
- Variantes por site/arquitetura e imagem multiarch fixada por digest.
- Helm e scripts permanecem como instrumentos de bootstrap, diagnóstico e recuperação do laboratório.

Definition of done do primeiro ciclo

1. Commit em repositório de implantação dispara reconciliação no NMI.
2. A xApp registra, assina E2SM-KPM e recebe uma indicação válida.
3. Rollback por Git remove a versão problemática e restaura a anterior.
4. Status e falha ficam visíveis sem acesso manual aos nós.

16. Laboratório guiado: criar, publicar e testar uma xApp

O roteiro completo está em docs/STUDENT_XAPP_LAB.md; esta seção resume a jornada que o aluno deve conseguir explicar.

OPEN RAN POC - UNB

Próximo marco: laboratório guiado para alunos

Experimentos dos alunos

- 1 Criar uma xApp pelo scaffold e escrever o algoritmo.
- 2 Publicar imagem multiarch fixada por digest.
- 3 Publicar Team Blueprint e PackageVariant no Porch.
- 4 Entregar por Flux no NMI e testar a variante CIC.
- 5 Executar UE2/UE3 e correlacionar o KPI no Grafana.
- 6 Evoluir para KPI por UE, rádio real e E2SM-RC.

Critério de sucesso

Um aluno cria, publica e implanta uma xApp; gera tráfego com um perfil UE; e explica a evidência KPM no Grafana.

Tutorial pronto; o próximo experimento será criado e apresentado pelos alunos.

17

Figura 9 — Próximo marco: o aluno fecha o ciclo entre código, pacote, tráfego UE e evidência KPM.

1. Criar uma branch e executar `./scripts/new-kpm-xapp.sh <nome> DRB.UETHPDI <período-ms>`.
2. Implementar uma mudança pequena no `indication_callback`: média móvel, limiar, estado ou contador.
3. Publicar a imagem multiarch e substituir o marcador pelo digest do índice amd64/arm64.
4. Publicar o Team Blueprint no Porch, criar a PackageVariant NMI e aprová-la.
5. Aplicar a identidade Flux restrita e aguardar a Kustomization ficar Ready.
6. Gerar tráfego com `demo-ue-lab.sh ue2` ou `ue3` e correlacionar o resultado no Grafana.

Critério de aceite do aluno

Pod Ready; registro no AppMgr; uma assinatura ativa; contadores RMR/KPM crescendo; HTTP 200 pelo UE; série visível no Prometheus/Grafana; rollback reproduzível.

16.1 Simulação multi-UE

Três assinantes foram cadastrados, mas o rádio virtual atual usa um perfil por vez no único enlace ZMQ.

- UE1, UE2 e UE3 representam três assinantes/identidades, mas usam em sequência o único enlace ZMQ da bancada.
- Não escalar srsUE para três réplicas: o DU oferece um único par de portas RF virtuais.
- O KPI DRB.UETpDI atual é agregado no O-DU; ele prova carga do rádio, mas ainda não identifica o IMSI responsável.

Perfil	IMSI	Uso
ue1	001010000000001	referência
ue2	001010000000002	experimento A
ue3	001010000000003	experimento B
Demonstração em um comando No control plane NMI: <code>sudo env KUBECONFIG=/etc/kubernetes/admin.conf ./scripts/demo-ue-lab.sh ue2</code> . Repita com ue3; espere attach, endereço 10.45.0.x, HTTP 200 e estados idle/active/busy.		

17. Conclusão para a apresentação

A POC já é uma plataforma de laboratório distribuída, automatizada e observável.

Hoje

NMI com 5 nós amd64, CIC com 2 nós ARM64, Nephio/Porch, Flux, Open5GS, RAN, RIC, três xApps e três perfis UE.

Próximo marco

Um aluno cria uma nova xApp, publica o pacote, implanta no NMI e explica no Grafana o efeito do tráfego UE.

Depois

KPI por UE, rádio e SIM físicos, novas estratégias de xApp e experimentos com E2SM-RC para artigos.

Mensagem final

O valor do laboratório não está em imitar produção: está em permitir que alunos mudem uma hipótese, executem uma simulação e obtenham evidência reproduzível da RAN.

Apêndice A — Fontes e rastreabilidade

As afirmações sobre o ambiente foram separadas das referências oficiais de produto.

Repositório e evidência operacional

- README.md; docs/VERSIONS.md; docs/NMI_DEPLOYMENT.md; docs/OBSERVABILITY.md; docs/STATUS.md; docs/LEARNINGS.md.
- helm/5g-core/values.yaml; helm/ran/values.yaml; helm/near-rt-ric/values.yaml.
- xapps/python/kpm_mon_xapp.py; xapps/python/lib/xAppBase.py; xapps/python/lib/e2sm_kpm_module.py.
- ansible/roles/deploy_xapp/tasks/main.yml.
- docs/STUDENT_XAPP_LAB.md; docs/UE_SIMULATION_LAB.md; scripts/demo-ue-lab.sh; scripts/demo-load-watch.sh.
- Validação funcional em 31/08/2026: NMI 5/5, CIC 2/2, 3 assinantes, 3 xApps, Flux nos dois sites, UE2/UE3 com HTTP 200 e KPM.

Documentação oficial Nephio

- Visão geral: [link oficial](#)
- Arquitetura: [link oficial](#)
- Porch: [link oficial](#)
- Instalação multi-VM: [link oficial](#)
- Release notes R6: [link oficial](#)
- Exemplo Flux em workload cluster: [link oficial](#)

O-RAN Software Community

- Subscription Manager: [repositório oficial](#)
- E2 Termination: [repositório oficial](#)
- Routing Manager: [repositório oficial](#)
- RMR: [repositório oficial](#)
- xApp Python framework: [repositório oficial](#)

Apêndice B — Glossário rápido

Termo	Significado no projeto
CU/DU	Funções centralizada e distribuída da gNB
E2	Interface entre Near-RT RIC e nós/funções RAN
E2SM-KPM	Modelo de serviço E2 para indicadores de desempenho
xApp	Aplicação executada sobre o Near-RT RIC
RMR	Barramento de mensagens do RIC
N2	Interface de controle entre gNB/CU-CP e AMF
F1-C	Interface de controle entre CU e DU
Multus	CNI que adiciona interfaces secundárias aos pods
Nephio	Automação declarativa/GitOps de funções de rede em Kubernetes
Porch	Servidor de pacotes kpt e orquestração de configuração do Nephio
Management cluster	Cluster que hospeda Nephio e seus controladores
Workload cluster	Cluster que executa as funções de rede